



Kong API Gateway & Konnect

Workshop Oficial: Ejercicio 001

Edición GitOps & Observabilidad Avanzada (SigNoz)

Duración Estimada: 130 minutos



Índice de Contenidos

- [Introducción](#)
- [A. Pre-requisitos y convención de puertos \(10 min\)](#)
 - [A.1 Pre-requisitos mínimos](#)
 - [A.2 Variables para comandos \(Ajustadas a tu entorno local\)](#)
- [B. Secuencia de ejercicios \(120 min\)](#)
 - [B.1 Preparación de Herramientas de Prueba \(5 min\)](#)
 - [B.2 Gobierno: separación por Control Plane \(External/Internal/Global\) \(10 min\)](#)
 - [B.3 Creación de Data Planes y Convención de Puertos \(15 min\)](#)
- [B.4 Observabilidad Global: Promoción de Estándares a través del Control Plane Global \(10 min\)](#)
 - [B.5 Gobierno: Teams y RBAC \(5 min\)](#)
- [B.6 Sincronización del estado base de APIs \(10 min\)](#)
- [B.7 Control de exposición: matching por método HTTP \(5 min\)](#)
- [B.8 Seguridad: autenticación con Key Auth \(10 min\)](#)
- [B.9 Seguridad: autorización con ACL \(10 min\)](#)
- [B.10 Control de tráfico: rate limiting diferenciado por Consumer \(10 min\)](#)
- [B.11 Transformación: Response Transformer \(5 min\)](#)
- [B.12 Transformación: Request Transformer \(OPCIONAL\) \(10 min\)](#)
- [B.13 Trazabilidad: request/correlation identifier \(5 min\)](#)
- [B.14 Seguridad adicional: IP Restriction \(5 min\)](#)
- [B.15 Observabilidad: Konnect Analytics Explorer \(5 min\)](#)

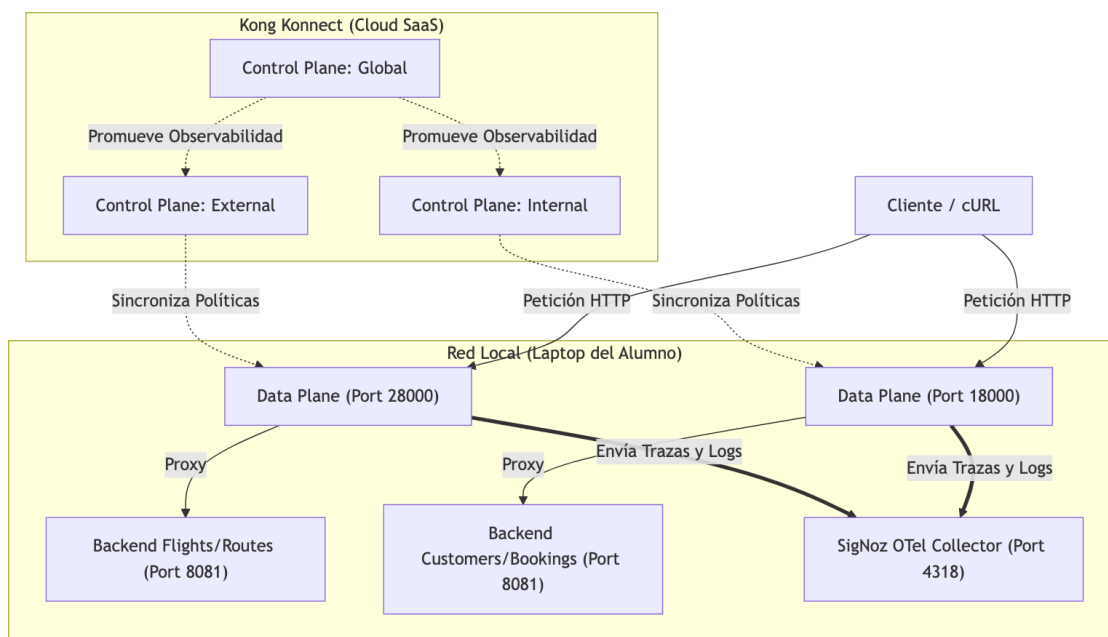
Introducción

En este laboratorio integrador construiremos desde cero una plataforma API Gateway gestionada declarativamente (GitOps) con observabilidad avanzada.

A lo largo del taller, emularemos un ecosistema distribuido donde:

- **Konnect (SaaS)** actuará como nuestro panel de control centralizado, delegando la administración a distintos equipos de desarrollo (External e Internal) y a un equipo de Plataforma (Global).
- **Data Planes Locales** procesarán el tráfico real, consumiendo APIs mock (vuelos, rutas, clientes, reservas).
- **SigNoz (OpenTelemetry)** funcionará como nuestro pilar de observabilidad unificada, recibiendo trazas y logs directamente desde el gateway para validar el impacto de cada política de seguridad que apliquemos.

El siguiente diagrama detalla la arquitectura que implementaremos:



A continuación, encontrarás la guía paso a paso para cumplir con el alcance del **Ejercicio 001** de Kong. En este ejercicio aplicaremos políticas de exposición, seguridad, control de tráfico, transformaciones y observabilidad, gestionando la configuración a través de Kong Konnect y deck.

Fase 0: Instalación de Prerrequisitos

Antes de iniciar el entorno local, asegúrate de tener instaladas las herramientas necesarias (Docker, deck, Terraform, jq, etc.). Hemos preparado un script automatizado para facilitar este proceso. Ejecuta el comando correspondiente a tu sistema operativo:

```
# Para Mac/Linux
./scripts/install_prereqs.sh

# Para Windows (CMD)
.\scripts\install_prereqs.bat

# Para Windows (PowerShell)
.\scripts\install_prereqs.ps1
```

Validación: El script terminará imprimiendo las versiones de todas las herramientas instaladas. Verifica que no haya mensajes de error marcados en rojo. (Nota para Windows: es posible que debas reiniciar tu consola después de este paso para que el comando `deck` esté disponible globalmente).

A. Pre-requisitos y convención de puertos (10 min)

A.1 Pre-requisitos mínimos

- Konnect accesible desde el navegador (usuario con permisos para administrar Control Planes, Services, Routes y Plugins).

- Backends de ejemplo disponibles ejecutándose como contenedores.
- Herramienta de prueba: cURL o Insomnia.
- CLI de `terraform` instalado localmente (se usará para crear la infraestructura base como Control Planes y Teams).
- `deck` instalado localmente (se usa para exportar/promover configuración de APIs).
- Verificar que tienen los contenedores backend de Kong Air funcionando. Se puede usar el script provisto para levantarlos (solamente el backend, la configuración de Kong se hará más adelante):

```
# Para Mac/Linux
cd ./ejercicio-001
./scripts/setup.sh

# Para Windows (CMD)
cd .\ejercicio-001
.\scripts\setup.bat
```

Validación: Comprueba que el backend simulado se haya iniciado correctamente.

- **En Mac/Linux:** `curl -s -i http://localhost:8081/anything/flights | head -n 1`
- **En Windows:** `curl.exe -s -i http://localhost:8081/anything/flights | findstr "HTTP/1.1"` (Debería retornar `HTTP/1.1 200 OK`)

- **Instalar SigNoz como contenedor local** para la sección de observabilidad. Se provee un script que clona y levanta los contenedores oficiales:

```
# Para Mac/Linux
./scripts/setup-signoz.sh

# Para Windows (CMD)
.\scripts\setup-signoz.bat
```

A.2 Variables para comandos (Ajustadas a tu entorno local)

Antes de operar con `deck` y Terraform, necesitas obtener tus credenciales y los identificadores de tu entorno en Konnect.

1. Generar tu Konnect Personal Access Token (KPAT)

El KPAT es necesario para que las herramientas de automatización se autenticuen contra Konnect.

1. Ingresa a la interfaz web de **Kong Konnect**.
2. Haz clic en tu avatar o menú de usuario (esquina superior derecha) y selecciona **Personal Access Tokens**.
3. Haz clic en **Generate Token**.
4. Asigne un nombre descriptivo (ej. `workshop-deck`) y una expiración (ej. 30 días).
5. Copia el token generado (comienza con `kpat_`). **¡Guárdalo en un lugar seguro porque no podrás volver a verlo!**

2. Obtener la URL Base (KONNECT_ADDR)

- **KONNECT_ADDR:** Es la URL base de tu región en Konnect. Si tu cuenta está en US, es `https://us.api.konghq.com` . Si está en Europa, es `https://eu.api.konghq.com` .

2. Configurar las variables en tu terminal

Ajusta los valores con la información que acabas de obtener, y elige un prefijo único para ti (por ejemplo, tu nombre, iniciales, o alias) para evitar colisiones con otros compañeros. Luego ejecuta los comandos en tu terminal:

Para Mac/Linux (Bash/Zsh):

```
export KCONNECT_ADDR="https://us.api.konghq.com"
export KCONNECT_TOKEN="kpat_***"
export PARTICIPANT_PREFIX="tu_nombre_aqui"
```

Para Windows (CMD):

```
set KCONNECT_ADDR=https://us.api.konghq.com
set KCONNECT_TOKEN=kpat_***
set PARTICIPANT_PREFIX=tu_nombre_aqui
```

3. Configurar deck CLI (Archivo .deck.yaml)

Para que la herramienta `deck` se conecte por defecto a la nube de Konnect (en la región correcta) y no busque un gateway local, generaremos su archivo de configuración:

Para Mac/Linux (Bash/Zsh):

```
cat <<EOF > ~/.deck.yaml
konnect-token: $KCONNECT_TOKEN
konnect-addr: $KCONNECT_ADDR
EOF
```

Para Windows (CMD):

```
echo konnect-token: %KCONNECT_TOKEN% > "%USERPROFILE%\deck.yaml"
echo konnect-addr: %KCONNECT_ADDR% >> "%USERPROFILE%\deck.yaml"
```

Validación: Ejecuta `deck gateway ping`. Nota: Si utilizas `deck` v1.40+, es **totalmente esperado** que recibas el mensaje `Error: control plane not found: default`. Esto confirma que `deck` logró autenticarse en Konnect exitosamente pero aún no tienes un Control Plane llamado "default". Puedes ignorar el error y continuar a la Fase B de forma segura.

B. Secuencia de ejercicios (120 min)

B.1 Preparación de Herramientas de Prueba (5 min)

Objetivo: preparar el entorno de pruebas local importando las colecciones necesarias.

1. Abre **Insomnia** (o Postman) y crea un nuevo proyecto (por ejemplo, `Workshop KongAir`).
2. Haz clic en **Create** -> **Import** y selecciona el archivo de especificación OpenAPI (`kongair-openapi.yaml`).

3. Repite el proceso de **Import** para cargar la colección de pruebas (`kongair-postman-collection.json`).
4. *Nota:* Aún no ejecutaremos peticiones, ya que primero construiremos la infraestructura (Control Planes y Data Planes) y publicaremos las APIs. Estas colecciones las utilizaremos a partir del paso B.6 para validar el flujo de tráfico.

B.2 Gobierno: separación por Control Plane (External/Internal/Global) (10 min)

Objetivo: mostrar segmentación de configuración y permisos por dominio funcional, automatizando la infraestructura con **Terraform**.

En lugar de crear los Control Planes de forma manual, usaremos Infraestructura como Código (IaC) para asegurar repetibilidad y trazabilidad.

1. Abre tu terminal y navega al directorio `terraform` de este ejercicio:

```
cd ./ejercicio-001/terraform
```

2. Inicializa Terraform y valida el código:

```
terraform init
terraform validate
```

3. Aplica los cambios proporcionando tus variables (la primera vez). Esto creará los Control Planes y Equipos con tu prefijo para evitar colisiones:

Para Mac/Linux:

```
terraform apply -var="konnnect_token=$KONNECT_TOKEN" -
var="participant_prefix=$PARTICIPANT_PREFIX"
```

Para Windows (CMD):

```
terraform apply -var="konnnect_token=%KONNECT_TOKEN%" -
var="participant_prefix=%PARTICIPANT_PREFIX%"
```

4. En Konnect: Confirma visualmente en **Gateway Manager** la creación de los tres Control Planes.
5. **Registro de Identificadores:** Ahora que los Control Planes han sido creados, haz clic en cada uno de ellos y observa la URL en tu navegador (el ID es el texto con formato UUID `12345678-...`). Configura estas variables en tu terminal, ya que las usarás para interactuar con tu infraestructura:

Para Mac/Linux (Bash/Zsh):

```
export CP_EXTERNAL_NAME="${PARTICIPANT_PREFIX}_KongAir_External"
export CP_EXTERNAL_ID="*****-****-****-****-*****"

export CP_INTERNAL_NAME="${PARTICIPANT_PREFIX}_KongAir_Internal"
export CP_INTERNAL_ID="*****-****-****-****-*****"
```

```
export CP_GLOBAL_NAME="${PARTICIPANT_PREFIX}_KongAir_Global"
export CP_GLOBAL_ID="*****-****-****-****-*****"
```

Para Windows (CMD):

```
set CP_EXTERNAL_NAME=${env:PARTICIPANT_PREFIX}_KongAir_External
set CP_EXTERNAL_ID=*****-****-****-****-*****

set CP_INTERNAL_NAME=${env:PARTICIPANT_PREFIX}_KongAir_Internal
set CP_INTERNAL_ID=*****-****-****-****-*****

set CP_GLOBAL_NAME=${env:PARTICIPANT_PREFIX}_KongAir_Global
set CP_GLOBAL_ID=*****-****-****-****-*****
```

B.3 Creación de Data Planes y Convención de Puertos (15 min)

Una vez que los Control Planes han sido creados (Paso B.2), deben conectarse a instancias físicas o virtuales que procesen el tráfico, conocidas como Data Planes.

En un laboratorio que emula tres ámbitos distintos (External, Internal y Global), cada Data Plane debe ejecutarse como un proceso o contenedor independiente y exponer puertos distintos en la máquina (host) para evitar conflictos de red.

*[!TIP] **Vía Rápida Automatizada:** Si prefieres no realizar la configuración manual en la UI, puedes levantar ambos Data Planes automáticamente utilizando los scripts provistos. Primero, genera los certificados y extrae los endpoints. (Asegúrate de tener la variable `PARTICIPANT_PREFIX` configurada).*

Para Mac/Linux:

```
python3 scripts/generate_certs.py
./scripts/start_dps_signoz_net.sh
```

Para Windows (CMD):

```
python scripts\generate_certs.py
.\scripts\start_dps_signoz_net.bat
```

Para Windows (PowerShell):

```
python scripts\generate_certs.py
.\scripts\start_dps_signoz_net.ps1
```

Si usas esta vía rápida, puedes ignorar el resto del paso B.3 y saltar directamente al paso B.4.

Vía Manual (desde la UI):

1. En **Gateway Manager** (Kconnect), ingresa al Control Plane `KongAir_External`.
2. Dirígete a **Data Plane Nodes** y haz clic en **New Data Plane Node**.
3. Selecciona tu plataforma (por ejemplo, Linux / Docker) y copia el script de inicio generado.

4. **Modificación del Script:** Pega el script en un editor de texto (como Notepad o VSCode) ANTES de ejecutarlo, para realizar dos ajustes críticos:

- o **Puertos:** Ajusta los puertos expuestos (`-p`) para que coincidan con la tabla de la convención de abajo (ej. `-p 28000:8000`).
- o **Observabilidad:** Inyecta las variables `KONG_TRACING_INSTRUMENTATIONS` y `KONG_TRACING_SAMPLING_RATE` para habilitar el traceo.

Ejemplo de cómo debería quedar tu comando para el Control Plane External:

```
docker run -d --name kong-dp-external \
-e "KONG_ROLE=data_plane" \
-e "KONG_DATABASE=off" \
-e "KONG_VITALS=on" \
-e "KONG_VITALS_TTL_DAYS=732" \
-e "KONG_CLUSTER_MTLS=pci" \
-e "KONG_CLUSTER_CONTROL_PLANE=xxxxxxxxx.us.cp.konghq.com:443" \
-e "KONG_CLUSTER_SERVER_NAME=xxxxxxxxx.us.cp.konghq.com" \
-e "KONG_CLUSTER_TELEMETRY_ENDPOINT=xxxxxxxxx.us.tp.konghq.com:443" \
-e "KONG_CLUSTER_TELEMETRY_SERVER_NAME=xxxxxxxxx.us.tp.konghq.com" \
-e "KONG_CLUSTER_CERT=-----BEGIN CERTIFICATE-----..." \
-e "KONG_CLUSTER_CERT_KEY=-----BEGIN EC PRIVATE KEY-----..." \
-e "KONG_TRACING_INSTRUMENTATIONS=all" \
-e "KONG_TRACING_SAMPLING_RATE=1.0" \
-p 28000:8000 \
-p 28443:8443 \
-p 28100:8100 \
kong/kong-gateway:3.14
```

5. **Convención de Puertos a respetar:**

Ámbito	Proxy HTTP (host)	Proxy HTTPS (host)	Metrics / Status (host)
External	-p 28000:8000	-p 28443:8443	-p 28100:8100
Internal	-p 18000:8000	-p 18443:8443	-p 18100:8100
Global	N/A	N/A	

6. Ejecuta tu comando modificado para el **External** y verifica que aparezca "In Sync" en la UI de Konnect.

7. (Opcional) Repite el proceso generando un nuevo certificado en el Control Plane

`KongAir_Internal` , y adapta el script usando los puertos correspondientes (`18000` , `18443` y `18100`).

B.4 Observabilidad Global: Promoción de Estándares a través del Control Plane Global (10 min)

Objetivo: demostrar el valor arquitectónico de usar un Control Plane Global como repositorio central de políticas (sin Data Planes) y promover la política de observabilidad (SigNoz) a otros Control Planes mediante `deck gateway apply` .

1. Utilizaremos el archivo `archivos-deck/00-b4-estado-base-global.yaml`. Puedes inspeccionar su contenido, que **solo** contiene los plugins transversales, sin rutas ni servicios:

```
_format_version: "3.0"
plugins:
  - name: opentelemetry
    config:
      header_type: w3c
      traces_endpoint: http://host.docker.internal:4318/v1/traces
      logs_endpoint: http://host.docker.internal:4318/v1/logs
      access_logs:
        endpoint: http://host.docker.internal:4318/v1/logs
      metrics:
        enable_request_metrics: true
        enable_latency_metrics: true
        enable_bandwidth_metrics: true
        endpoint: http://host.docker.internal:4318/v1/metrics
      resource_attributes:
        service.name: kong-api-gateway
        deployment.environment: workshop
```

2. **Sincroniza el estándar al Control Plane Global:** Esto guardará la configuración centralizada, aunque el Global CP no ejecute tráfico.

Para Mac/Linux:

```
deck gateway sync archivos-deck/00-b4-estado-base-global.yaml --konnnect-token
"$KONNECT_TOKEN" --konnnect-control-plane-name "$CP_GLOBAL_NAME"
```

Para Windows (CMD):

```
deck gateway sync archivos-deck/00-b4-estado-base-global.yaml --konnnect-token
%KONNECT_TOKEN% --konnnect-control-plane-name %CP_GLOBAL_NAME%
```

3. **Promoción de Estándares (Previsualización):** En el mundo real, asume que el Control Plane External ya tendría decenas de APIs y rutas configuradas por los desarrolladores. Usa el comando `diff` para previsualizar la inyección del estándar global. Verás que deck te indica que *añadirá* el plugin `opentelemetry` de forma segura. (Si usáramos `sync` en un entorno real, ¡borraríamos todo el trabajo previo de ese equipo!).

Para Mac/Linux:

```
deck gateway diff archivos-deck/00-b4-estado-base-global.yaml --konnnect-token
"$KONNECT_TOKEN" --konnnect-control-plane-name "$CP_EXTERNAL_NAME"
```

Para Windows (CMD):

```
deck gateway diff archivos-deck/00-b4-estado-base-global.yaml --konnnect-token
%KONNECT_TOKEN% --konnnect-control-plane-name %CP_EXTERNAL_NAME%
```


4. **Aplicar (Merge) el estándar al External e Internal:** Usa el comando `apply` para inyectar este plugin transversal en tus CPs de manera segura.

Para Mac/Linux:

```
deck gateway apply archivos-deck/00-b4-estado-base-global.yaml --kconnect-  
token "$KCONNECT_TOKEN" --kconnect-control-plane-name "$CP_EXTERNAL_NAME"  
deck gateway apply archivos-deck/00-b4-estado-base-global.yaml --kconnect-  
token "$KCONNECT_TOKEN" --kconnect-control-plane-name "$CP_INTERNAL_NAME"
```

Para Windows (CMD):

```
deck gateway apply archivos-deck/00-b4-estado-base-global.yaml --kconnect-  
token %KCONNECT_TOKEN% --kconnect-control-plane-name %CP_EXTERNAL_NAME%  
deck gateway apply archivos-deck/00-b4-estado-base-global.yaml --kconnect-  
token %KCONNECT_TOKEN% --kconnect-control-plane-name %CP_INTERNAL_NAME%
```

5. **Nota Importante:** Aún no podemos generar tráfico ni ver resultados en SigNoz porque los Control Planes están vacíos. En el paso B.6 cargaremos las APIs y haremos la primera validación.

B.5 Gobierno: Teams y RBAC (5 min)

Objetivo: demostrar delegación de administración y aislamiento de permisos.

(Nota: Terraform ya aplicó esta configuración en el paso anterior, aquí solo realizaremos la validación)

1. En Konnect: Ve a **Organization -> Teams**. Verifica la creación de los dos equipos: `External Developers` e `Internal Developers`.
2. Haz clic en el equipo `External Developers` y revisa sus roles. Verás que tiene asignado `Control Plane Admin` únicamente para `KongAir_External`.
3. Haz lo mismo con `Internal Developers` y valida que solo tienen permisos sobre `KongAir_Internal`.
4. Con esto, confirmamos que los permisos están correctamente acotados por Control Plane y que la infraestructura base está lista para recibir las APIs.

B.6 Sincronización del estado base de APIs (10 min)

Objetivo: poblar los Control Planes External e Internal con sus respectivos servicios base usando deck, respetando el aislamiento de dominios.

1. Abre tu terminal en el directorio raíz del ejercicio (`ejercicio-001`).
2. **IMPORTANTE:** Como en el paso B.4 inyectamos una política global de observabilidad, si hacemos un `sync` directamente con nuestro archivo de APIs, borraríamos ese plugin global. Para mantener una arquitectura GitOps limpia e incremental (sin copiar y pegar el plugin en todos los archivos), utilizaremos el comando `deck file merge` para combinar las políticas globales con las APIs locales antes de sincronizar.
3. Combina la política global con las APIs públicas (`flights` , `routes`) y sincroniza al Control Plane **External**:

Para Mac/Linux:

```
deck file merge archivos-deck/00-b4-estado-base-global.yaml archivos-deck/01-b6-estado-base-external.yaml -o release-external.yaml
deck gateway sync release-external.yaml --konnekt-token "$KONNECT_TOKEN" --konnekt-control-plane-name "$CP_EXTERNAL_NAME"
```

Para Windows:

```
deck file merge archivos-deck/00-b4-estado-base-global.yaml archivos-deck/01-b6-estado-base-external.yaml -o release-external.yaml
deck gateway sync release-external.yaml --konnekt-token %KONNECT_TOKEN% --konnekt-control-plane-name %CP_EXTERNAL_NAME%
```

4. Combina la política global con las APIs privadas (`customers` , `bookings`) y sincroniza al Control Plane **Internal**:

Para Mac/Linux:

```
deck file merge archivos-deck/00-b4-estado-base-global.yaml archivos-deck/01-b6-estado-base-internal.yaml -o release-internal.yaml
deck gateway sync release-internal.yaml --konnekt-token "$KONNECT_TOKEN" --konnekt-control-plane-name "$CP_INTERNAL_NAME"
```

Para Windows:

```
deck file merge archivos-deck/00-b4-estado-base-global.yaml archivos-deck/01-b6-estado-base-internal.yaml -o release-internal.yaml
deck gateway sync release-internal.yaml --konnekt-token %KONNECT_TOKEN% --konnekt-control-plane-name %CP_INTERNAL_NAME%
```

5. En Konnect: Verifica en **Gateway Manager** que `KongAir_External` tenga 2 servicios, y `KongAir_Internal` tenga otros 2 servicios.
6. Ahora que las APIs existen, valida que puedes consumir las rutas recién sincronizadas a través de sus respectivos Data Planes (External e Internal):

Para Mac/Linux:

```
curl -i http://localhost:28000/flights
curl -i http://localhost:18000/customers
```

Para Windows:

```
curl.exe -i http://localhost:28000/flights
curl.exe -i http://localhost:18000/customers
```

7. **Primera Validación de Observabilidad:** Abre SigNoz navegando a `http://localhost:8080` (la primera vez que ingreses se te pedirá crear una cuenta de administrador local). Una vez iniciada la sesión, ve a la sección **Traces** y verifica que aparezca la traza de tu llamada exitosa a `/flights` . A partir de ahora, usaremos SigNoz para ver el efecto de todas nuestras políticas de seguridad.

8. **Generación de Tráfico Masivo:** Para apreciar el valor del stack de observabilidad, generemos carga enviando una ráfaga de 100 peticiones a cada uno de los 4 servicios disponibles:

Para Mac/Linux:

```
for i in {1..100}; do curl -s -o /dev/null http://localhost:28000/flights;
done
for i in {1..100}; do curl -s -o /dev/null http://localhost:28000/routes;
done
for i in {1..100}; do curl -s -o /dev/null http://localhost:18000/customers;
done
for i in {1..100}; do curl -s -o /dev/null http://localhost:18000/bookings;
done
```

Para Windows:

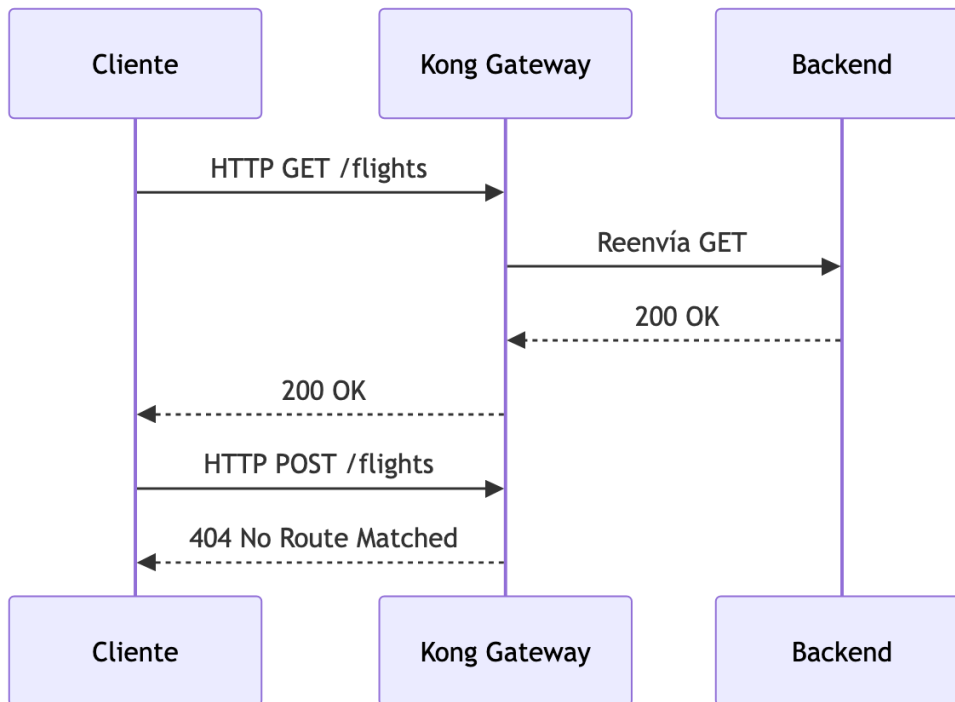
```
1..100 | ForEach-Object { curl.exe -s -o NUL http://localhost:28000/flights }
1..100 | ForEach-Object { curl.exe -s -o NUL http://localhost:28000/routes }
1..100 | ForEach-Object { curl.exe -s -o NUL http://localhost:18000/customers }
}
1..100 | ForEach-Object { curl.exe -s -o NUL http://localhost:18000/bookings }
}
```

9. **Análisis Avanzado (Dashboards y Topología):** Para visualizar este tráfico, exploraremos tres capacidades fundamentales del stack de observabilidad:

- o **Acceso General:** Abre tu navegador e ingresa a la interfaz de SigNoz en `http://localhost:8080`.
- o **Traces (Trazas detalladas):** Navega en el menú izquierdo a **Traces** (o ve a `http://localhost:8080/traces`). Podrás ver el listado de cada una de las peticiones masivas generadas. Haz clic en una de ellas para acceder al gráfico de Gantt (Waterfall) y observar exactamente cuántos milisegundos demoró Kong en procesar la petición versus el tiempo real del backend simulado.
- o **Service Map (Topología):** Haz clic en **Service Map** en el menú izquierdo (o ve a `http://localhost:8080/services`). Verás un grafo dinámico generado automáticamente. Este mapa mostrará a tu API Gateway (`kong-api-gateway`) como el nodo central recibiendo conexiones de los clientes y enrutando tráfico hacia el backend. Notarás líneas animadas mostrando el flujo y la Tasa de Peticiones por Segundo (RPS).
- o **Dashboards (Métricas):** Ve a la sección **Dashboards** (o `http://localhost:8080/dashboards`). En lugar de crear los gráficos desde cero, importaremos un dashboard preconfigurado:
 1. Haz clic en el botón **New Dashboard** (o "Create Dashboard") en la parte superior derecha y selecciona **Import JSON**.
 2. Selecciona y carga el archivo `kong-gateway.json` que se encuentra en la carpeta raíz del taller (`ejercicio-001`).
 3. Haz clic en **Load JSON**.
 4. Automáticamente verás un panel completo (Kong API Gateway) con métricas críticas (RPS, Latencia, Códigos de Estado, Ancho de Banda). Podrás observar claramente los fuertes picos correspondientes al momento en que ejecutaste tus scripts generadores de carga.

B.7 Control de exposición: matching por método HTTP (5 min)

Objetivo: reducir la superficie expuesta restringiendo los métodos en una Route mediante configuración declarativa.



1. Utilizaremos el archivo `archivos-deck/02-b7-metodos.yaml`. Puedes inspeccionar cómo se agregaron los cambios:

```
services:
- name: flights
  url: http://host.docker.internal:8081/anything/flights
  routes:
- name: flights-route
  paths:
- /flights
  methods:
- GET
```

2. Sincroniza los cambios:

Para Mac/Linux:

```
deck gateway sync archivos-deck/02-b7-metodos.yaml --kconnect-token
"$KONNECT_TOKEN" --kconnect-control-plane-name "$CP_EXTERNAL_NAME"
```

Para Windows:

```
deck gateway sync archivos-deck/02-b7-metodos.yaml --konnnect-token
%KONNECT_TOKEN% --konnnect-control-plane-name %CP_EXTERNAL_NAME%
```

3. Probar el método permitido y uno no permitido:

Para Mac/Linux:

```
curl -i http://localhost:28000/flights
curl -i -X POST http://localhost:28000/flights
```

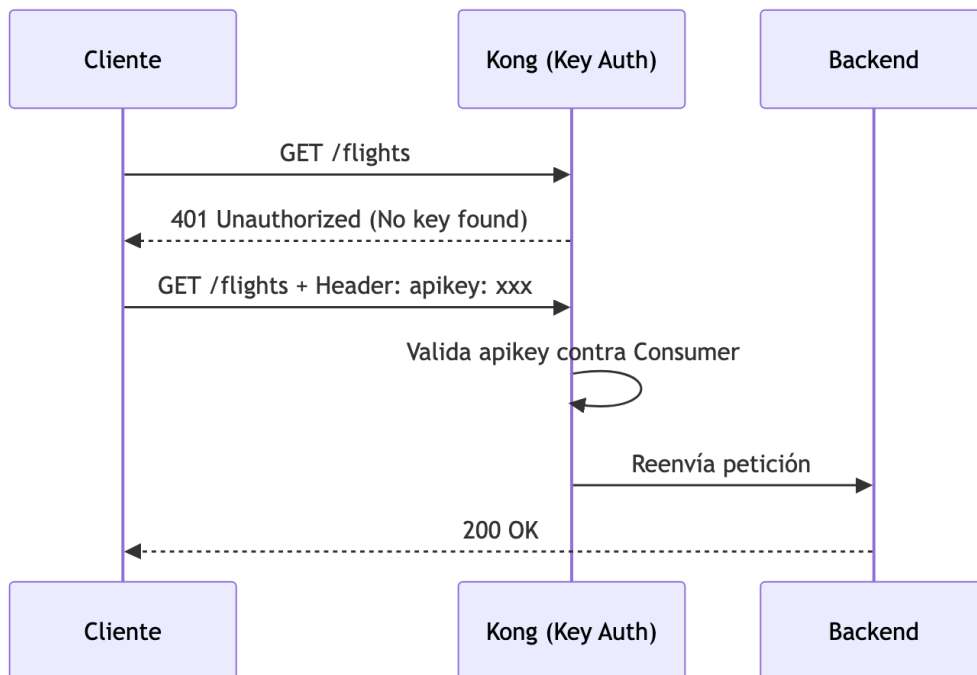
Para Windows:

```
curl.exe -i http://localhost:28000/flights
curl.exe -i -X POST http://localhost:28000/flights
```

Resultado esperado: 200 OK para el GET y 404 No Route Matched para el POST.

B.8 Seguridad: autenticación con Key Auth (10 min)

Objetivo: centralizar la autenticación en el Gateway declarando plugins y Consumers.



1. Utilizaremos el archivo `archivos-deck/03-b8-key-auth.yaml`. Puedes inspeccionar cómo se agregaron los cambios:

```
services:
- name: flights
```

```

url: http://host.docker.internal:8081/anything/flights
plugins:
  - name: key-auth
    config:
      key_names:
        - apikey
  routes: # ... (resto de la configuración)

consumers:
  - username: App-External
    keyauth_credentials:
      - key: external-secret-123
  - username: App-Internal
    keyauth_credentials:
      - key: internal-secret-123

```

2. Sincroniza los cambios:

Para Mac/Linux:

```

deck gateway sync archivos-deck/03-b8-key-auth.yaml --konnekt-token
"$KONNECT_TOKEN" --konnekt-control-plane-name "$CP_EXTERNAL_NAME"

```

Para Windows:

```

deck gateway sync archivos-deck/03-b8-key-auth.yaml --konnekt-token
%KONNECT_TOKEN% --konnekt-control-plane-name %CP_EXTERNAL_NAME%

```

3. Probar sin credencial y con credencial:

Para Mac/Linux:

```

curl -i http://localhost:28000/flights
curl -i http://localhost:28000/flights -H "apikey: external-secret-123"

```

Para Windows:

```

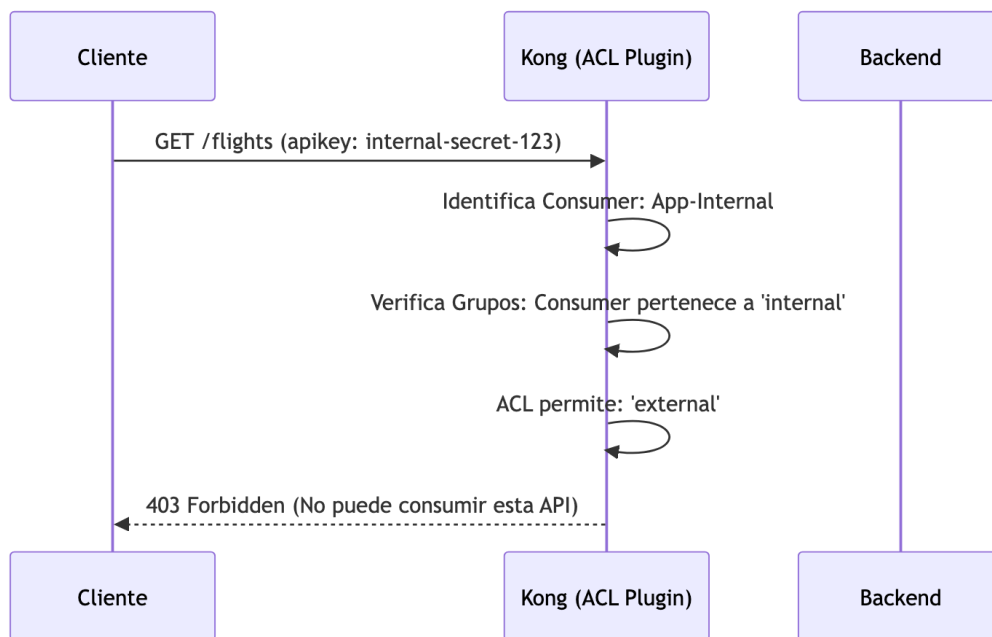
curl.exe -i http://localhost:28000/flights
curl.exe -i http://localhost:28000/flights -H "apikey: external-secret-123"

```

Resultado esperado: 401 Unauthorized sin llave y 200 OK con llave. (Verifica en SigNoz cómo Kong registra la traza del 401).

B.9 Seguridad: autorización con ACL (10 min)

Objetivo: diferenciar autenticación de autorización y restringir acceso declarando grupos ACL.



1. Utilizaremos el archivo `archivos-deck/04-b9-acl.yaml` . Puedes inspeccionar cómo se agregaron los cambios:

```
services:
  - name: flights
    plugins:
      - name: key-auth
        config:
          key_names: [apikey]
      - name: acl
        config:
          allow:
            - external
# ...
consumers:
  - username: App-External
    acls:
      - group: external
    keyauth_credentials:
      - key: external-secret-123
  - username: App-Internal
    acls:
      - group: internal
    keyauth_credentials:
      - key: internal-secret-123
```

2. Sincroniza los cambios:

Para Mac/Linux:

```
deck gateway sync archivos-deck/04-b9-acl.yaml --konnnect-token  
"$KONNECT_TOKEN" --konnnect-control-plane-name "$CP_EXTERNAL_NAME"
```

Para Windows:

```
deck gateway sync archivos-deck/04-b9-acl.yaml --konnnect-token  
%KONNECT_TOKEN% --konnnect-control-plane-name %CP_EXTERNAL_NAME%
```

3. Probar con ambas credenciales:

Para Mac/Linux:

```
curl -i http://localhost:28000/flights -H "apikey: internal-secret-123"  
curl -i http://localhost:28000/flights -H "apikey: external-secret-123"
```

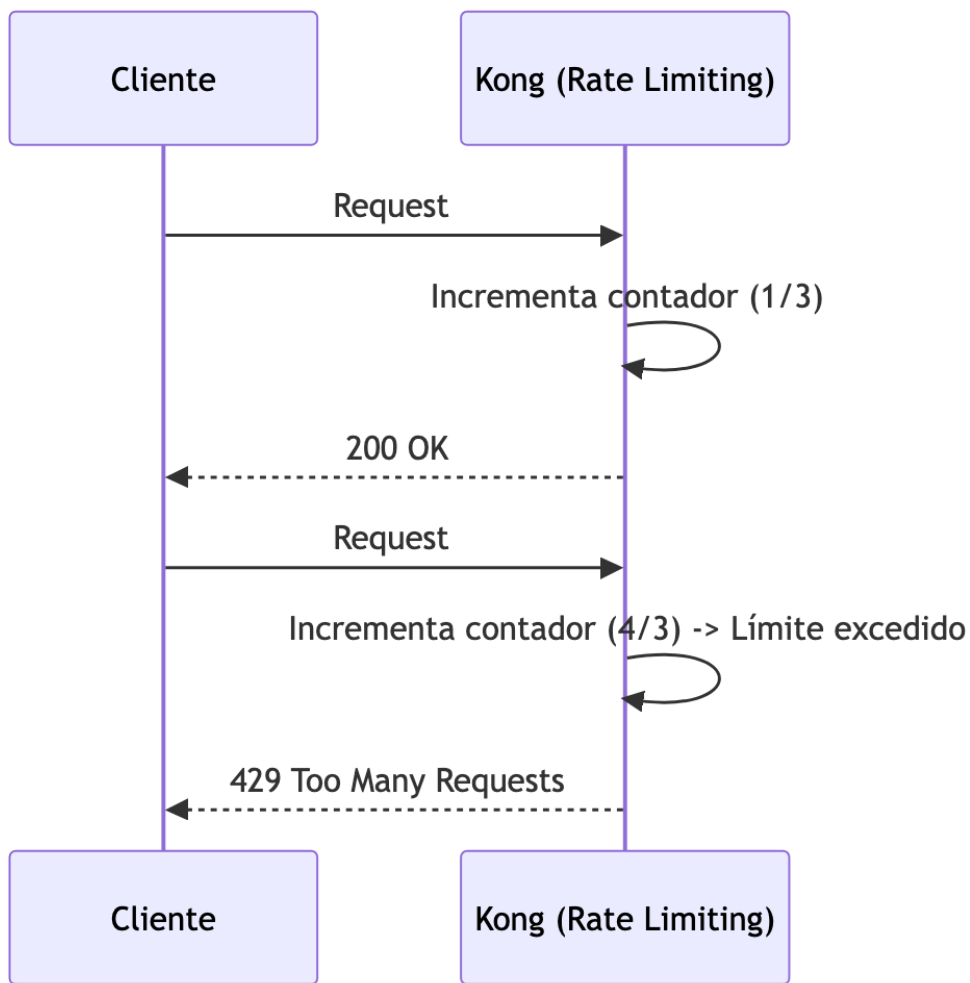
Para Windows:

```
curl.exe -i http://localhost:28000/flights -H "apikey: internal-secret-123"  
curl.exe -i http://localhost:28000/flights -H "apikey: external-secret-123"
```

Resultado esperado: 403 Forbidden para App-Internal y 200 OK para App-External. (Abre SigNoz y observa cómo la política ACL bloqueó el request).

B.10 Control de tráfico: rate limiting diferenciado por Consumer (10 min)

Objetivo: aplicar límites distintos por perfil de consumo anidando plugins en Consumers.



1. Utilizaremos el archivo `archivos-deck/05-b10-rate-limiting.yaml` . Puedes inspeccionar cómo se agregaron los cambios:

```
consumers:
- username: App-External
  plugins:
  - name: rate-limiting
    config:
      minute: 20
      policy: local
  # ... acls y credentials ...
- username: App-Internal
  plugins:
  - name: rate-limiting
    config:
      minute: 3
      policy: local
  # ... acls y credentials ...
```

2. Sincroniza los cambios:

Para Mac/Linux:

```
deck gateway sync archivos-deck/05-b10-rate-limiting.yaml --konnnect-token "$KONNECT_TOKEN" --konnnect-control-plane-name "$CP_EXTERNAL_NAME"
```

Para Windows:

```
deck gateway sync archivos-deck/05-b10-rate-limiting.yaml --konnnect-token %KONNECT_TOKEN% --konnnect-control-plane-name %CP_EXTERNAL_NAME%
```

3. Generar tráfico para probar los límites:

Para Mac/Linux:

```
for i in {1..6}; do curl -s -o /dev/null -w "%{http_code}\n" http://localhost:28000/flights -H "apikey: internal-secret-123"; done
```

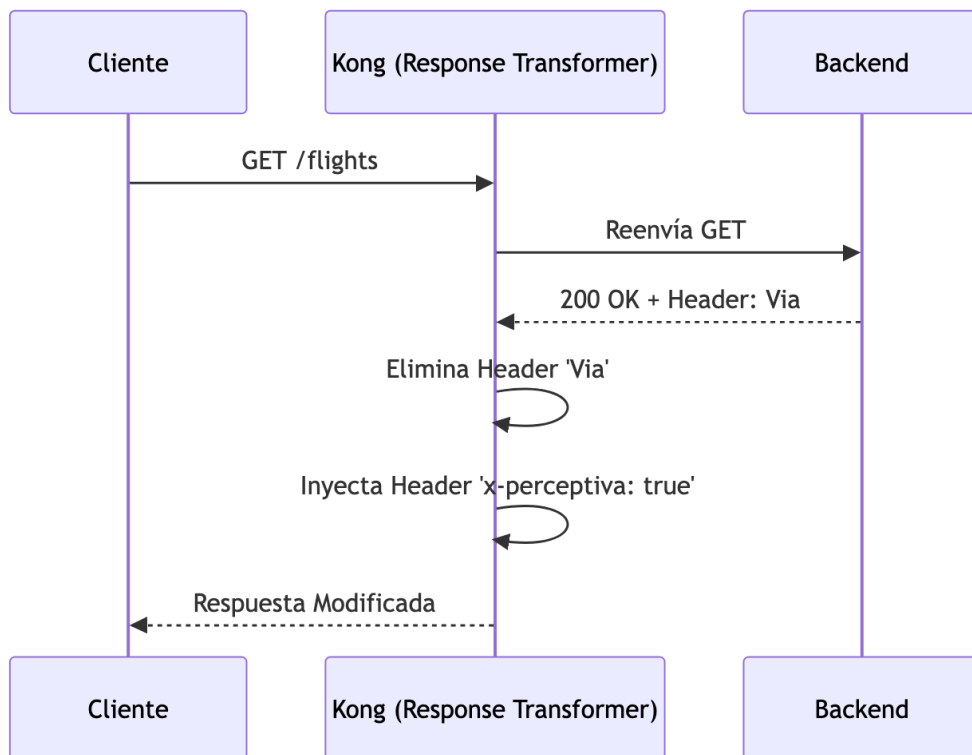
Para Windows:

```
1..6 | ForEach-Object { curl.exe -s -o NUL -w "%{http_code}\n" http://localhost:28000/flights -H "apikey: internal-secret-123" }
```

Verás que App-Internal llega a `429 Too Many Requests` muy rápido. Abre SigNoz para visualizar la explosión de requests bloqueados (429).

B.11 Transformación: Response Transformer (5 min)

Objetivo: inyectar headers de respuesta sin modificar el backend.



1. Utilizaremos el archivo `archivos-deck/06-b11-response-transformer.yaml`. Puedes inspeccionar cómo se agregaron los cambios:

```

routes:
  - name: flights-route
    paths:
      - /flights
    methods:
      - GET
    plugins:
      - name: response-transformer
        config:
          add:
            headers:
              - "x-perceptiva: true"
          remove:
            headers:
              - "Via"
  
```

2. Sincroniza los cambios:

Para Mac/Linux:

```

deck gateway sync archivos-deck/06-b11-response-transformer.yaml --konnnect-
token "$KONNECT_TOKEN" --konnnect-control-plane-name "$CP_EXTERNAL_NAME"
  
```

Para Windows:

```
deck gateway sync archivos-deck/06-b11-response-transformer.yaml --konnnect-token %KONNECT_TOKEN% --konnnect-control-plane-name %CP_EXTERNAL_NAME%
```

3. Probar y verificar los headers de respuesta:

Para Mac/Linux:

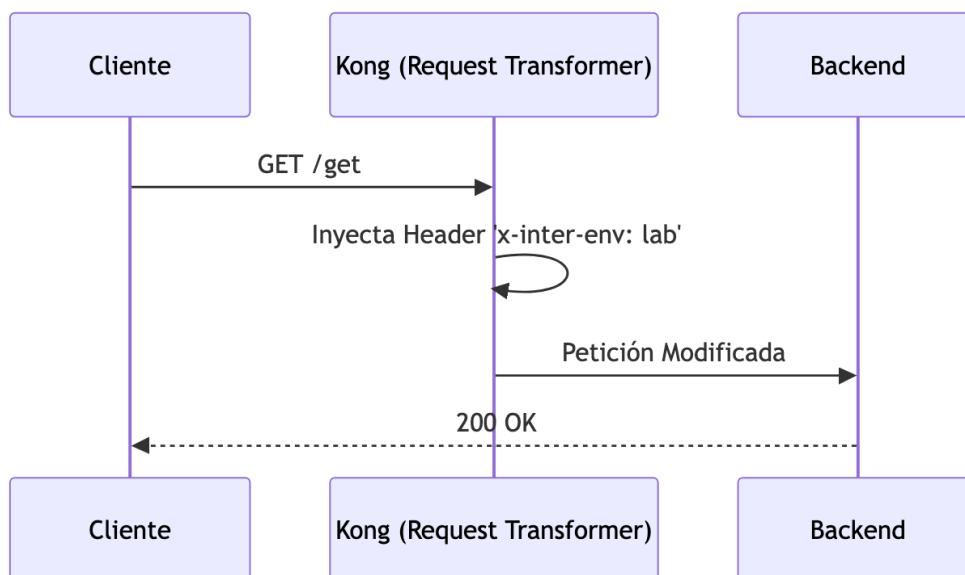
```
curl -i http://localhost:28000/flights -H "apikey: external-secret-123"
```

Para Windows:

```
curl.exe -i http://localhost:28000/flights -H "apikey: external-secret-123"
```

B.12 Transformación: Request Transformer (OPCIONAL) (10 min)

Objetivo: enriquecer requests hacia el upstream creando nuevos servicios y rutas de prueba.



1. Utilizaremos el archivo `archivos-deck/07-b12-request-transformer.yaml`. Puedes inspeccionar cómo se agregaron los cambios:

```
- name: get-test
  url: http://host.docker.internal:8081/get
  routes:
    - name: get-route
      paths:
        - /get
      plugins:
```

```
- name: request-transformer
  config:
    add:
      headers:
        - "x-inter-env: lab"
```

2. Sincroniza los cambios:

Para Mac/Linux:

```
deck gateway sync archivos-deck/07-b12-request-transformer.yaml --konnnect-
token "$KONNECT_TOKEN" --konnnect-control-plane-name "$CP_EXTERNAL_NAME"
```

Para Windows:

```
deck gateway sync archivos-deck/07-b12-request-transformer.yaml --konnnect-
token %KONNECT_TOKEN% --konnnect-control-plane-name %CP_EXTERNAL_NAME%
```

3. Probar y validar en el JSON de respuesta que el backend recibió `X-Inter-Env: lab` :

Para Mac/Linux:

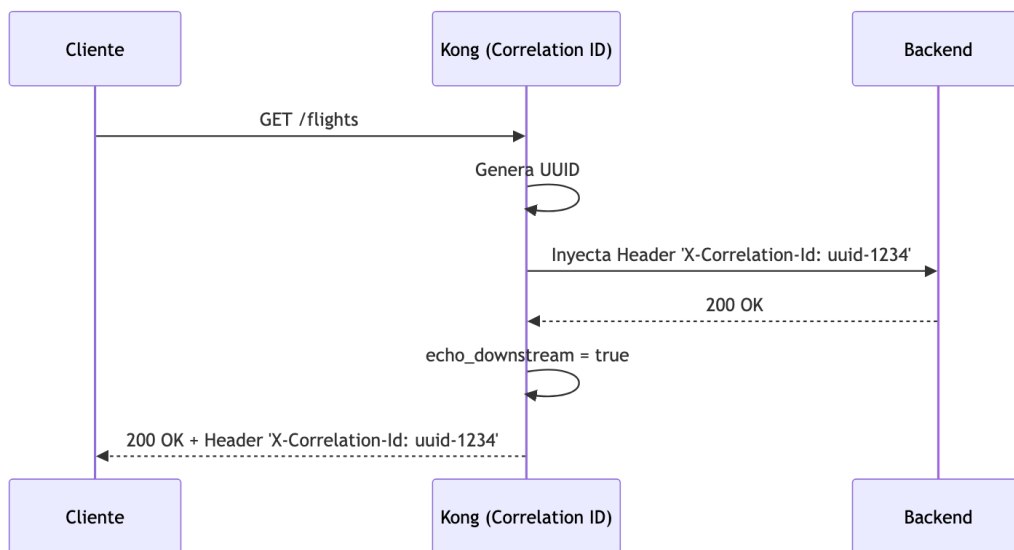
```
curl -s http://localhost:28000/get
```

Para Windows:

```
curl.exe -s http://localhost:28000/get
```

B.13 Trazabilidad: request/correlation identifier (5 min)

Objetivo: inyectar IDs de correlación globales agregando un plugin a nivel de Control Plane.



1. Utilizaremos el archivo `archivos-deck/08-b13-correlation-id.yaml` . Puedes inspeccionar cómo se agregaron los cambios:

```

_format_version: "3.0"
plugins:
  - name: correlation-id
    config:
      header_name: x-correlation-id
      echo_downstream: true
      generator: uuid
services:
  # ...

```

2. Sincroniza los cambios:

Para Mac/Linux:

```

deck gateway sync archivos-deck/08-b13-correlation-id.yaml --konnekt-token
"$KONNECT_TOKEN" --konnekt-control-plane-name "$CP_EXTERNAL_NAME"

```

Para Windows:

```

deck gateway sync archivos-deck/08-b13-correlation-id.yaml --konnekt-token
%KONNECT_TOKEN% --konnekt-control-plane-name %CP_EXTERNAL_NAME%

```

3. Ejecutar una llamada y observar el header `X-Correlation-Id` en la respuesta:

Para Mac/Linux:

```

curl -i http://localhost:28000/flights -H "apikey: external-secret-123"

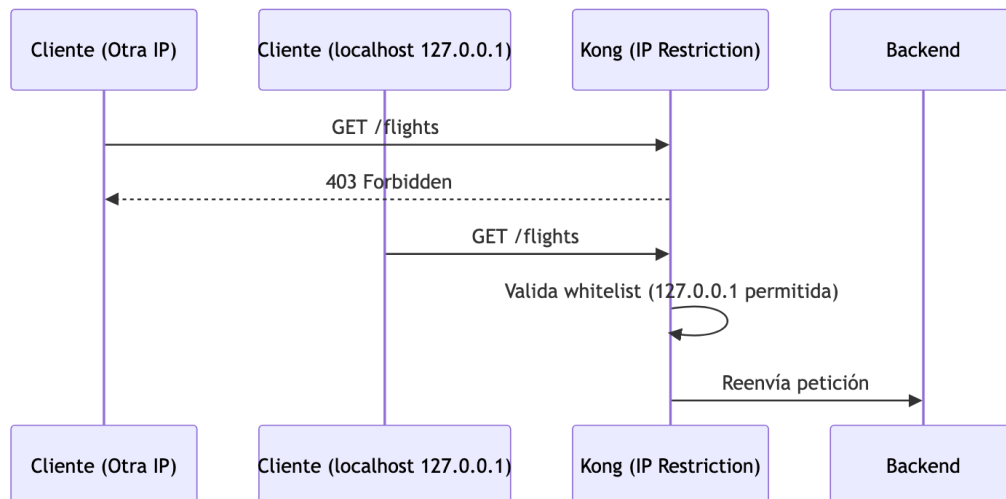
```

Para Windows:

```
curl.exe -i http://localhost:28000/flights -H "apikey: external-secret-123"
```

B.14 Seguridad adicional: IP Restriction (5 min)

Objetivo: aplicar restricción por IP directamente en una ruta.



1. Utilizaremos el archivo `archivos-deck/09-b14-ip-restriction.yaml`. Puedes inspeccionar cómo se agregaron los cambios:

```
plugins:
  - name: response-transformer
    # ...
  - name: ip-restriction
    config:
      allow:
        - 127.0.0.1
```

2. Sincroniza los cambios:

Para Mac/Linux:

```
deck gateway sync archivos-deck/09-b14-ip-restriction.yaml --konnekt-token "$KONNECT_TOKEN" --konnekt-control-plane-name "$CP_EXTERNAL_NAME"
```

Para Windows:

```
deck gateway sync archivos-deck/09-b14-ip-restriction.yaml --konnekt-token %KONNECT_TOKEN% --konnekt-control-plane-name %CP_EXTERNAL_NAME%
```

3. Probar desde localhost y desde otra IP (si es posible) para ver el rechazo `403 Forbidden` . Revisa en SigNoz cómo la validación de IP corta la petición antes de llegar al upstream.

B.15 Observabilidad: Konnect Analytics Explorer (5 min)

Objetivo: visualizar tráfico, errores y segmentación sin herramientas adicionales.

1. En Konnect, ve a **Analytics -> Explorer**. Selecciona el rango de los últimos 15 minutos.
2. Filtra por Service/Route `flights` .
3. Identifica patrones generados: `401` (sin apikey), `403` (ACL/IP restriction), `429` (rate limiting), `200` (tráfico permitido).

¡Felicidades! Has completado el laboratorio implementando un Gateway moderno, gestionado declarativamente (GitOps) y con observabilidad avanzada.